

BetterSearch

Semantic content search: retrieval by **meaning** rather than by shared words.

Search a content library for `Norman conquest` and get back articles about *1066*, the *Battle of Hastings* and the *Domesday Book* — none of which contain the phrase you typed.

Here is the same idea in a library catalogue. One reader, one phrasing, the same 74 records, and the only difference is which button is pressed.

Catalogue search — 0 results.

 Northfield Libraries

[My account](#) [Events](#) [Help](#)

RS

 R. Sandiford

Full catalogue ▾

Search

Catalogue search

Search by meaning

Try: [learning to be present](#) [gentle crime novels, nothing too gory](#) [grewal](#) [getting my toddler to eat vegetables](#) [books to help me sleep better](#)

Home > Search results

Anywhere: learning to be present

Matched by **keywords** — records containing the words you typed.

Filter results

No results

< Previous Next >

Nothing in the catalogue matched those words.

The catalogue holds 74 items. A keyword search can only return a record that literally contains what you typed — if a reader's wording differs from the catalogue's, the item cannot be found at all.

Switch to **Search by meaning** to see what is actually here.

Search by meaning — 20 results.

 Northfield Libraries [My account](#) [Events](#) [Help](#)

learning to be present

Full catalogue ▼ **Search**

Catalogue search

Search by meaning

Try: [learning to be present](#) [gentle crime novels, nothing too gory](#) [grewal](#) [getting my toddler to eat vegetables](#) [books to help me sleep better](#)

Home > Search results

Anywhere: learning to be present

Matched by **meaning** — records about this subject, whatever words they use.

Filter results

Showing 1 - 10 of 20

[Previous](#) [Page 1](#) [Next](#)

Books / Events ▼

☐ Books 20

Availability ▼

☐ Available now 17

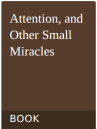
☐ Available online 2

Format ▼

☐ Book 14

☐ Large print 3

☐ eAudiobook 2

1.  **Attention, and Other Small Miracles**

Verhoeven, Sanne
Bellwether, 2017
No summary in this record.

[Contemplation](#) [Psychology](#)

Place reservation Available: **2** Copies: 4 · Fenwick Road

2.  **Living Lightly**

Not one of those twenty records contains the word *learning* or the word *present*. Eighteen of the twenty have no summary at all — nothing but a title, an author and a subject heading.

Twenty of seventy-four is a lot, and that is the honest state of it: the page returns a fixed top 20, a number measured against the 4,000-record corpus further down, not against this one. On a shelf this small the last few are thin. There is no corpus-independent right answer here — [see below](#) for what replaced the threshold that used to do this job, and why a number that looked sensible was the wrong one.

Why this exists

Keyword search (BM25, Postgres full-text, Elasticsearch's default) ranks by term overlap. If the user's words are not in the document, the document cannot be found — however obviously relevant it is. That is the entire failure mode this proof of concept addresses.

Semantic search embeds text as vectors positioned by meaning, so "Norman conquest" lands near "the invading army of Duke William" without sharing a single content word.

Install and run

```
pip install -e ".[dev,local,api]"          # local backend: no API key needed
bettersearch ingest --corpus data/corpus_seed.json
bettersearch compare "Norman conquest"      # the demonstration
bettersearch evaluate --per-query           # the measurement
```

Then for the side-by-side UI:

```
uvicorn api.main:app --reload               # http://127.0.0.1:8000
```

Nothing above needs an API key, a database, or network access after the first model download. A fresh clone has **no index** — `.bettersearch/` is gitignored, so `ingest` must run before any search.

For deployment, expected outputs and troubleshooting, see [RUNBOOK.md](#).

The catalogue prototype

The screenshots above come from a working page, not a mockup. It is the same library, the same index and the same `Searcher` — a catalogue-shaped front end over it, for showing the idea to people who will never read an nDCG table.

```
export BETTERSEARCH_INDEX_PATH=.bettersearch/catalogue
bettersearch ingest --corpus data/catalogue_library.json
uvicorn api.main:app --reload      # http://127.0.0.1:8000/catalogue
```

Three queries are worth trying, in this order:

1. **learning to be present** — the pair above. Catalogue search cannot return a record that does not contain the words; meaning-based search returns 20, led by *Attention, and Other Small Miracles*.
2. **gentle crime novels, nothing too gory** — a request phrased the way a reader actually asks. Keyword search finds three records and gets the point exactly backwards: it leads with *Nine Grams*, an organised-crime thriller, and picks up a book on spiritual life called *Nothing to Do* because the record contains the word "nothing". Meaning-based search opens with *The Knitting Circle Murders* and has *Tea, Cake and Arsenic* sixth, with *Nine Grams* pushed down to fifteenth.

This one is worth reading twice, because it used to say something weaker. Under the old MiniLM encoder *Nine Grams* still landed **second** — "crime novels" outweighed "nothing too gory" — and the honest conclusion was "better here, not magic". Swapping the encoder for `bge-base-en-v1.5` moved it to fifteenth. The negation is now being read. Nothing about the records or the query changed.

3. **grewal** — the failure that is easiest to miss, because it looks like it worked:

Anywhere: **grewal**

Matched by **keywords** — records containing the words you typed.

Filter results

Books / Events

☐ Books10

Availability

☐ Available now9

Format

☐ Book9☐ DVD1

Fiction / Non-fiction

☐ Non-fiction6☐ Fiction4

Publication date

☐ 2010s3☐ 2020s3☐ 1990s1☐ 2000s1

Location

☐ Ashcombe Branch3☐ Fenwick Road3

Showing 1 - 10 of 10

< PreviousPage 1Next >

1.

[Punjabi book]

[Punjabi book]

Grewal, Prem, 1931-1990

UUUU

No summary in this record.

Spiritual life

Place reservation

Available: 2Copies: 3Ashcombe Branch

2.

Nothing to Do

Nothing to Do

Grewal, Prem, 1931-1990

Wayfarer Press, UUUU

No summary in this record.

Spiritual life

Place reservation

Available: 3Copies: 5Fenwick Road

3.

The Knitting Circle Murders

The Knitting Circle Murders

Grewal, Ravi

Pennyfarthing, 2021

No summary in this record.

Detective and mystery stories

A surname is not an identity. Two unrelated authors share one, so keyword search files a crime novel in among books on spiritual life and looks perfectly healthy doing it. Notice the first record as well: no real title, no summary,

3

no publication date. That record is not a contrived example — a catalogue of any size is full of them, and there is nothing there for a keyword search to match.

The catalogue is a fictional service with invented records, so it can be shown around without passing as a real library. Rebuild it with `python scripts/build_catalogue.py`, and regenerate these screenshots with `python scripts/capture_screenshots.py` against a running server.

To send the page to someone, `docs/catalogue-standalone.html` opens on a double-click with no server and no Python — and it answers **anything you type**, not a list of questions decided in advance. The whole catalogue, its embedding vectors and the search itself are in the file: 4.8 MB, built against the **real 4,000-record catalogue** rather than the invented one.

Both lanes really run. Keyword search is BM25 over the shipped records, ported from `src/bettersearch/keyword.py` and reproducing the API's ranking exactly — 41 of 41 eval queries, identical ordering. Searching by meaning embeds your query in the browser with `transformers.js` and scores it against int8 corpus vectors by dot product.

The page runs the same model as the server, and says so. Corpus vectors and query must come from the same encoder, so the file ships `bge-base-en-v1.5` (768 dimensions, ~110 MB quantised, fetched once on first meaning search and then cached). Measured against the API on the same corpus: **93.5% top-20 overlap, and nDCG@10 of 0.645 against the server's 0.651**. What is left is a tail reshuffle among comparably relevant records, caused by int8 vectors and a quantised model rather than by a different one; `scripts/check_browser_parity.py` is where those numbers come from and explains what it cannot remove.

It shipped `bge-small` until September 2026, on the reasoning that nobody should be asked to download `bge-base` to look at a demo. The quantised build is 110 MB against small's 34 MB, so that reasoning was wrong about its own premise, and it was expensive: the demo you send someone ranked differently from the service you tested, silently, and was the weaker of the two.

Keyword search needs no download at all, and the page says so if the model cannot load. `RUNBOOK.md` has the build command.

Worth opening it on `learning to take better photographs` and switching modes. Meaning-based search returns five photography books. Catalogue search returns the same book first, then *Again, but Better* (a young-adult romance) and *Write better, speak better* (public speaking), because they contain "better" — and, at fifth, *I will take a nap!*, a picture book about pigs, because it contains "take". That is the difference the rest of this document measures, in a file anyone can open.

Searching the events alongside the books

A reader asking what to do about their CV wants Tuesday's job club at least as much as a book on interview technique, and a catalogue that only holds books cannot tell them. So the events are a second collection in the same index: one query reaches both, and they compete for the same result slots.

```
BETTERSEARCH_INDEX_PATH=.bettersearch/real-events \
  bettersearch ingest --corpus data/catalogue_real.json \
    --corpus data/events_northfield.json
```

`how to keep bees in a small garden` returns *Beekeeping for Beginners* third, among the books; `what to expect when you are pregnant` puts the Bump and Beyond drop-in third; `dog training for a new puppy` has the Dog Training Talk fourth.

The events are invented, so this demonstrates the idea and measures nothing about it. There is no public feed of library events, and writing 114 of them reintroduces exactly the flattery that got the 74-record catalogue

replaced by real Open Library data: the author of the queries also wrote the answers. What *is* worth checking is the plumbing, which can fail in two opposite directions — 4,000 books against 114 events means an event may never surface at all, and the worse failure is an event surfacing where nobody wants one.

```
python scripts/check_events.py
```

event-shaped queries reaching an event	12 of 12
known-item lookups showing an event	0 of 5
event results across the 41 book queries	9 of 820 slots

The decision this fed was written down before the script ran: *if events are invisible, retrieve the two collections separately and fuse by rank*. They are not invisible, so `reciprocal_rank_fusion` stays unused and the single index stands. Worth recording that the fallback was specified first — it is much easier to decide a fusion layer was necessary after watching it work.

Two things this turned up.

There was one miss under the old encoder: `learn to knit` put *Knit and Natter* at rank 49, behind 48 knitting books. An event only wins where the shelves are thin, and knitting is not thin. Swapping to `bge-base` fixed that one too — all twelve now reach an event — which is a second reminder that "the corpus cannot answer this" and "this encoder cannot find it" look identical from outside.

The other was a real defect. A weekly session is several records, one per date, so a query matching the session matches all of them: `something short I can finish in one sitting` filled its first five slots with five identical copies of a chair exercise class, on a cosine score of 0.30. Nothing in any record describes how long a book is, so in that vacuum "sitting" matched "sitting down" and won. Occurrences of one series now collapse to the best-ranked of them, and the card says *and 6 other dates* — which is how a reader thinks about it anyway: one thing that happens on Tuesdays, not seven things.

The three modes

MODE	HOW IT RANKS	GOOD AT	BLIND TO
keyword	BM25 term overlap	exact names, dates, figures, jargon	anything phrased differently
semantic	cosine over embeddings	paraphrase, concepts, questions	precise tokens it blurs together
hybrid	reciprocal rank fusion of both	most real traffic	—

`hybrid` is the mode you would normally reach for — though on this corpus it measurably loses to pure `semantic`; see the results below. Fusion uses **rank**, not raw score: BM25 scores are unbounded and cosine scores sit in [-1, 1], so blending the numbers directly needs normalisation constants that must be re-tuned whenever the corpus or model changes. Ranks are comparable by construction.

Measured results

Run `bettersearch evaluate` to reproduce. 112 documents, 32 hand-labelled queries, numpy index. These were measured with `all-MiniLM-L6-v2`, the encoder that was the default at the time; the default is now `bge-base-en-v1.5`, so re-running will not reproduce them exactly. The 4,000-record section below is the current measurement.

MODE	RECALL@5	MRR@10	NDCG@10
keyword	0.355	0.625	0.415
semantic	0.841	0.984	0.890
hybrid	0.624	0.826	0.741

Semantic retrieval more than doubles nDCG over keyword and wins **30 of the 32 queries** outright. On eight queries — including `Norman conquest`, `why does bread rise`, `father of genetics` and `vaccination` — keyword search scores exactly **0.000**: not a poor ranking, but no relevant document retrieved at any depth.

Two illustrative failures worth seeing in the UI:

- `Norman conquest` → BM25 returns **nothing**. No document contains either word.
- `why does bread rise` → BM25 returns **Inflation**, because that article says "a sustained *rise* in the general price level". Term overlap without meaning.

Hybrid loses here, and that is a real result

Fusing a strong retriever with a weak one produced a *worse* ranking than the strong one alone (0.741 vs 0.890). That is not a bug in the fusion — it is what RRF does when one input list is mostly empty or noisy, which is the case on an eval set deliberately weighted towards keyword-hostile phrasing.

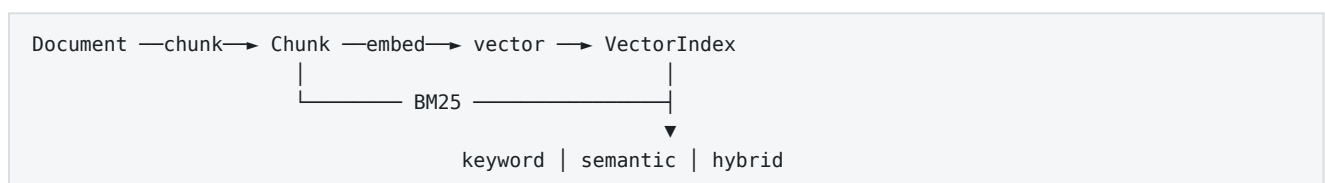
The honest reading: **ship semantic as the default for this kind of content**, and revisit `hybrid` only against a query mix with more exact-name, code and figure lookups, where BM25 earns its half of the fusion. Sample queries where keyword did contribute: `storing energy in a chemical form` (keyword 0.871 vs semantic 0.828) and `1066` (0.870 vs 0.876). Run `bettersearch evaluate --per-query` for the full breakdown.

Read this before quoting the numbers. The seed corpus and the eval labels were written by the same author, so the absolute values are indicative, not authoritative — the *gap between modes* is the signal. For a non-circular check, build a corpus you did not write:

```
python scripts/fetch_wikipedia.py --count 2000 --out data/corpus_wikipedia.json
bettersearch ingest --corpus data/corpus_wikipedia.json
bettersearch evaluate
```

Note also that the eval set is deliberately weighted towards keyword-hostile phrasing, because searching for words already in the text is the case keyword search already handles. Two control queries (`Bayeux Tapestry`, `evolution by natural selection`) use exact terminology, and BM25 is expected to win those.

Architecture



src/bettersearch/	
chunking.py	paragraph-aware splitting, overlap, content hashing
embeddings/	EmbeddingProvider protocol + local / OpenAI / Voyage
index/	VectorIndex protocol + numpy / pgvector
keyword.py	BM25 baseline
fusion.py	reciprocal rank fusion
search.py	Searcher – the public entry point
ingest.py	incremental ingestion
evaluate.py	Recall@5, MRR@10, nDCG@10
enrichment/	LLM enrichment of thin catalogue records
cli.py	
api/main.py	FastAPI wrapper - holds no retrieval logic
api/catalogue.py	catalogue-shaped presentation: facets, availability
web/index.html	developer comparison page
web/catalogue.html	the catalogue prototype

The library is framework-agnostic. `api/` and `cli.py` are thin callers, and neither is imported by the core.

For the full picture — every component, what each one does, how a request travels through them, and the interfaces to extend — see [docs/ARCHITECTURE.md](#), also available as a [PDF](#).

Rebuilding this on another engine? [docs/VESPA-HANDOVER.md](#) (PDF) is written for a team doing exactly that: what transfers, what to re-measure, and the traps. `scripts/score_rankings.py` scores any engine's output against the same 2,378 judgements, so a rebuild can be held to this one's bar rather than to a new one.

`web/index.html` is the view for working on retrieval rather than for showing anyone — all three modes on one screen, scored, so a change in chunking or model is visible immediately:

BetterSearch

The same query, run three ways over the same 112 documents. Keyword search ranks by shared words; semantic search ranks by meaning. Try a query whose wording appears nowhere in the content.

Search query

Search

Try: [Norman conquest](#) [why does bread rise](#) [the ship that sank on its maiden voyage](#) [father of genetics](#) [drugs that stop working](#)

Keyword search returned nothing for "Norman conquest". Semantic search found 5 results.

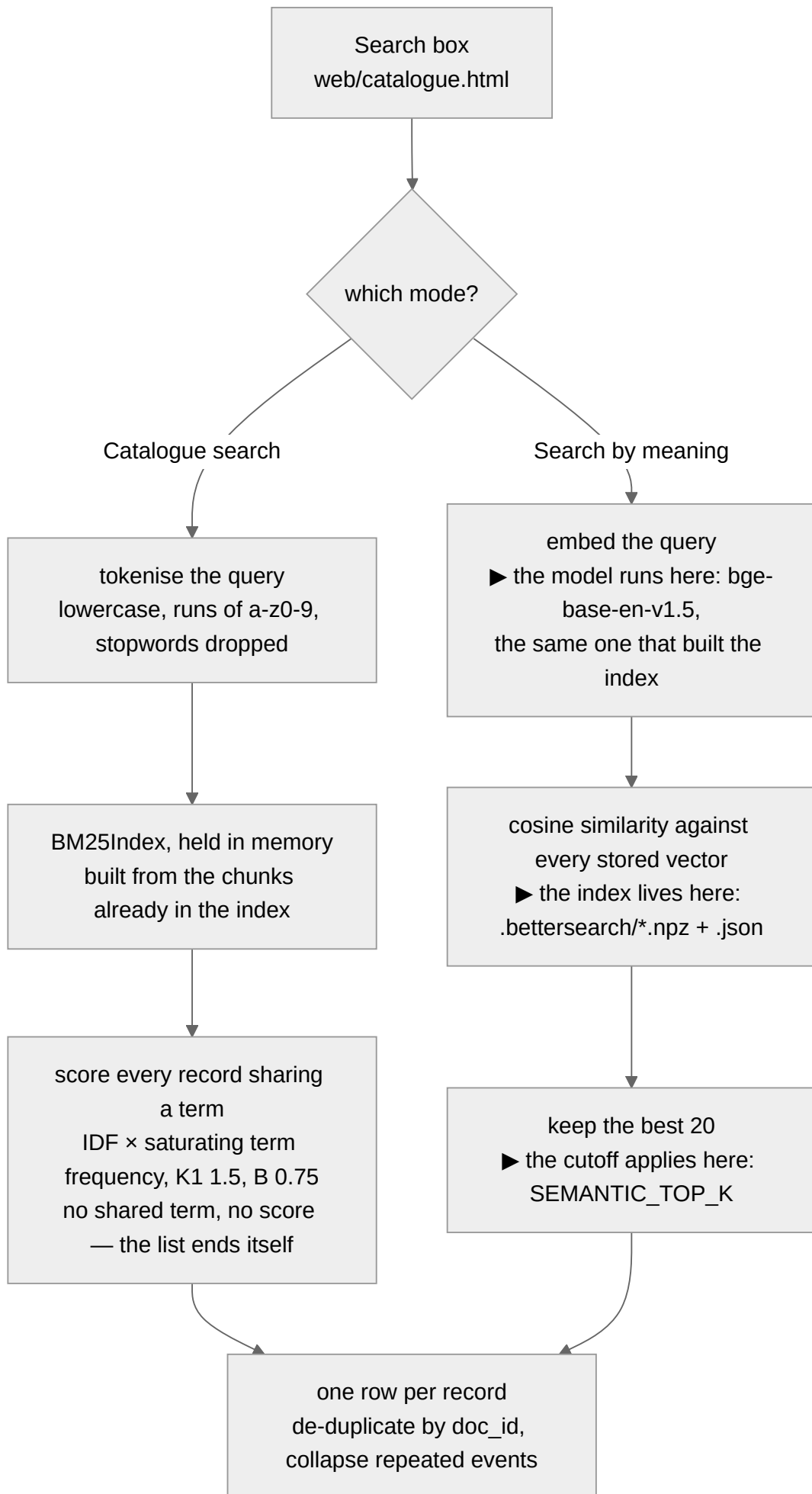
KEYWORD	SEMANTIC	HYBRID
BM25 term overlap. Cannot find a document that does not share your words.	Cosine similarity over embeddings. Matches meaning rather than vocabulary.	Reciprocal rank fusion of the other two columns.
<i>No results — no document shares these terms.</i>	0.428 The Battle of Hastings On 14 October 1066, an invading army under Duke William of Normandy met the English shield wall on Senlac Hill, a few miles inland from the coast of S...	0.016 The Battle of Hastings On 14 October 1066, an invading army under Duke William of Normandy met the English shield wall on Senlac Hill, a few miles inland from the coast of S...
	0.367 Harold Godwinson The last crowned Anglo-Saxon king of England, Harold was the most powerful nobleman in the kingdom before taking the throne in January 1066. His claim...	0.016 Harold Godwinson The last crowned Anglo-Saxon king of England, Harold was the most powerful nobleman in the kingdom before taking the throne in January 1066. His claim...
	0.328 Motte-and-Bailey Castles A raised earth mound topped by a wooden tower, with an enclosed courtyard below protected by a ditch and palisade. Cheap, fast to build, and defensibl...	0.016 Motte-and-Bailey Castles A raised earth mound topped by a wooden tower, with an enclosed courtyard below protected by a ditch and palisade. Cheap, fast to build, and defensibl...
	0.322 William, Duke of Normandy Born illegitimate in Falaise around 1028, William inherited his duchy as a child and spent two decades fighting to hold it. He claimed the English thr...	0.016 William, Duke of Normandy Born illegitimate in Falaise around 1028, William inherited his duchy as a child and spent two decades fighting to hold it. He claimed the English thr...
	0.316 The Battle of Britain Between July and October 1940 the Luftwaffe attempted to destroy British air defences as a precondition for invasion. Fighter Command was outnumbered ...	0.015 The Battle of Britain Between July and October 1940 the Luftwaffe attempted to destroy British air defences as a precondition for invasion. Fighter Command was outnumbered ...

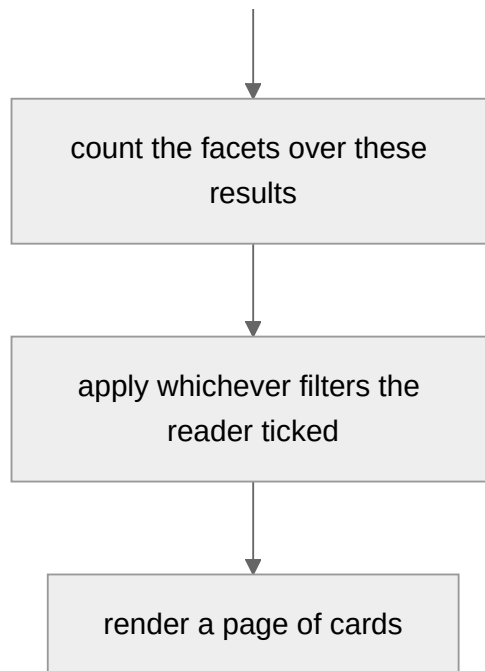
112 chunks indexed · provider local · model sentence-transformers/all-MiniLM-L6-v2@384 · 384 dims · numpy backend

```
$ bettersearch compare "Norman conquest" # the same thing at the terminal
```

What happens when you search

Both lanes search exactly the same indexed chunks — the BM25 index is built from whatever is already in the vector index — so the difference between the two columns is the retrieval method and nothing else.

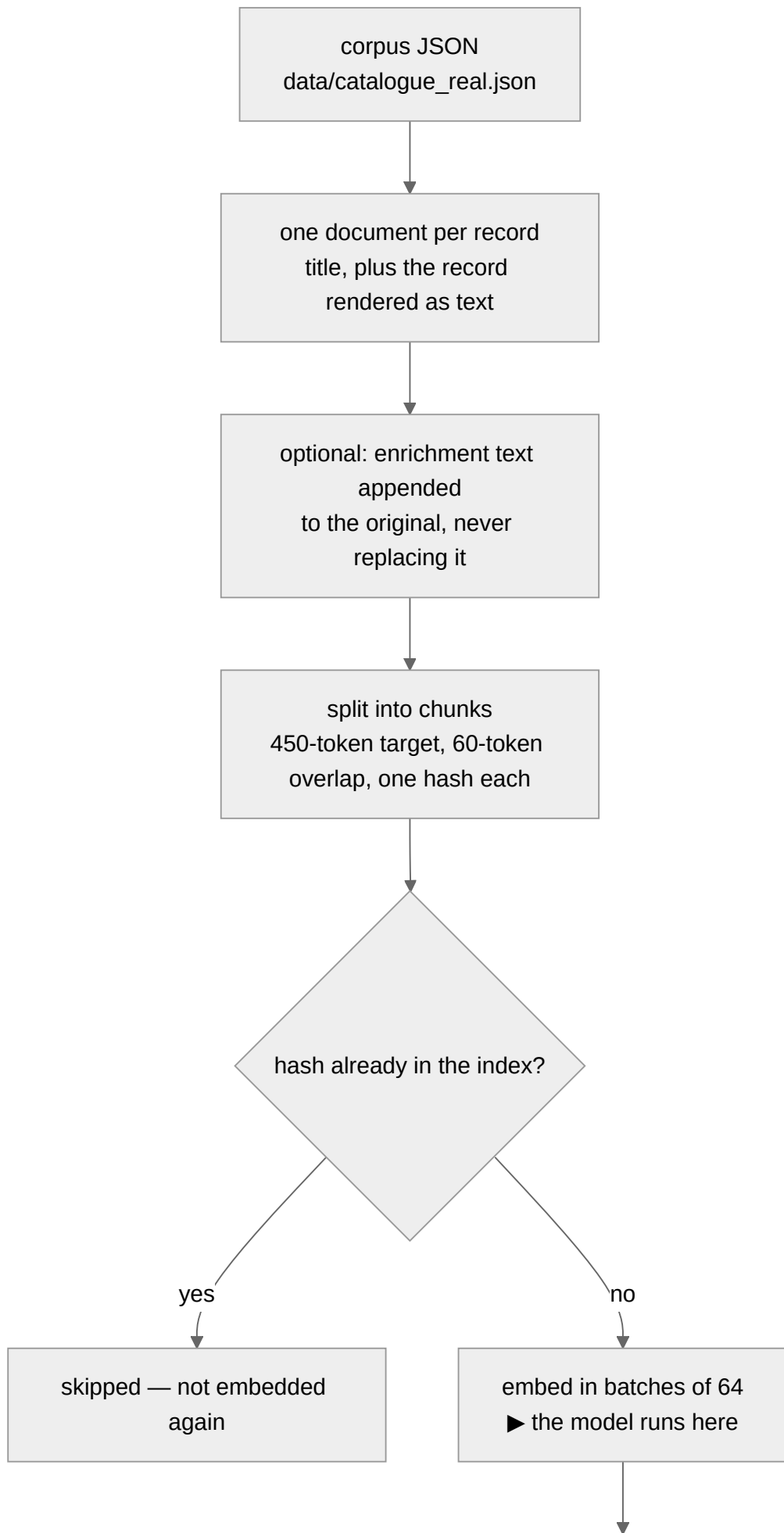


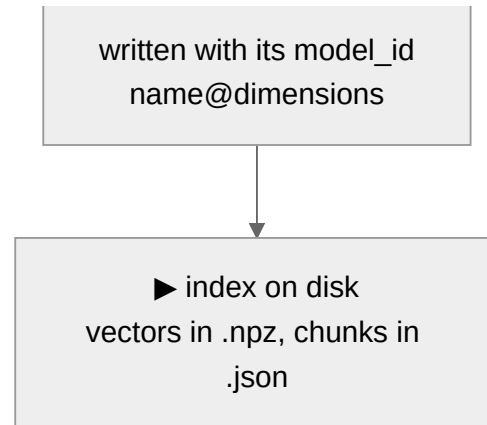


Typing in the box gives you one of two things. **Catalogue search** chops your words into terms, throws away the ones that carry no signal, and scores records by how unusual the shared words are and how often they appear — a record with nothing in common with your query simply never enters the list, which is why this mode can return three results or none. **Search by meaning** instead turns your sentence into a list of numbers using the same model the catalogue was indexed with, compares it against every record's numbers, and keeps the closest twenty; every record scores against every query, so something has to decide where the list stops, and that is the twenty. Both then converge: repeats are folded together, the facet counts are taken, your filters are applied, and what is left is drawn as cards.

Hybrid is not drawn because the catalogue page cannot reach it. The library has a third mode that fuses the two lanes by rank, and `/search` exposes it, but `api/catalogue.py` accepts only `keyword` or `semantic` — so the page in front of a reader has two lanes, not three. Why it was left out is measured, not assumed.

And how a record got into the index in the first place





Indexing is what makes searching by meaning possible, and it happens once rather than per search. Each record is turned into text, optionally given the extra descriptive sentences that enrichment produced, split into chunks and converted into numbers by the embedding model; those numbers are saved to disk alongside the name of the model that produced them. A chunk whose text has not changed keeps its existing numbers and is not re-embedded, so updating a catalogue costs only the records that actually moved. What the numbers buy is in the [measured results](#).

Configuration

Everything is a `BETTERSEARCH_*` environment variable; no code change is needed to switch provider or storage.

VARIABLE	DEFAULT	NOTES
<code>BETTERSEARCH_EMBEDDING_PROVIDER</code>	<code>local</code>	<code>local</code> <code>openai</code> <code>voyage</code>
<code>BETTERSEARCH_INDEX</code>	<code>numpy</code>	<code>numpy</code> <code>pgvector</code>
<code>BETTERSEARCH_INDEX_PATH</code>	<code>.bettersearch/index</code>	numpy backend only
<code>BETTERSEARCH_DATABASE_URL</code>	—	pgvector backend; falls back to <code>DATABASE_URL</code>
<code>BETTERSEARCH_EMBEDDING_DIMENSIONS</code>	<code>512</code>	Matryoshka width for API providers
<code>BETTERSEARCH_CHUNK_TOKENS</code>	<code>450</code>	target chunk size
<code>BETTERSEARCH_CHUNK_OVERLAP</code>	<code>60</code>	overlap between chunks
<code>BETTERSEARCH_LOCAL_MODEL</code>	<code>BAAI/bge-base-en-v1.5</code>	any sentence-transformers model
<code>BETTERSEARCH_LOCAL_TRUST_REMOTE_CODE</code>	<code>0</code>	required by some long-context encoders; runs code from the model repo

API keys are read from `OPENAI_API_KEY` / `VOYAGE_API_KEY` / `ANTHROPIC_API_KEY` at the point of use and never stored by this package. Only enrichment needs an Anthropic key; indexing and search do not.

Choosing the embedding model

The default is `BAAI/bge-base-en-v1.5`, and the reason is an input limit rather than a leaderboard. This repository started on `all-MiniLM-L6-v2`, which reads at most 256 tokens. That was fine on thin records and stopped being fine once enrichment made them longer: **79 chunks were cut off mid-text**, and a sentence the model never read cannot be retrieved by any query. `bge-base` reads 512 and truncated none of them. The docstring at the top of [src/bettersearch/embeddings/local.py](#) carries that figure, and the point behind it — the limit is architectural,

because the positional embeddings stop there, so more CPU or a bigger GPU makes a model faster and never able to read further.

Everything below is a drop-in swap. Sizes are approximate download sizes, and the token and dimension figures are the ones recorded in `SUGGESTED_MODELS`. Languages are taken from each model's own card.

MODEL	INPUT TOKENS	DIMENSIONS	DOWNLOAD	LANGUAGES	USE WHEN
<code>sentence-transformers/all-MiniLM-L6-v2</code>	256	384	80 MB	English	The smallest and quickest to embed. Check <code>truncated_chunks</code> after ingesting — 256 tokens is the ceiling that cost 79 chunks here.
<code>BAAI/bge-small-en-v1.5</code>	512	384	130 MB	English	Half the vector width of the default at the same window, and ~34 MB quantised. A lighter in-browser build, one <code>--model</code> flag away — but only worth it against a matching server.
<code>BAAI/bge-base-en-v1.5</code>	512	768	440 MB	English	The default , and what the served API runs. Its 512-token window covered every chunk in both corpora here.
<code>intfloat/e5-large-v2</code>	512	1024	1.3 GB	English	Same window as the default, one third wider vectors — so one third more index per record.
<code>nomic-ai/nomic-embed-text-v1.5</code>	8192	768	550 MB	English	Long records, at the default's vector width. Needs <code>BETTERSEARCH_LOCAL_TRUST_REMOTE_CODE=1</code> .
<code>Alibaba-NLP/gte-large-en-v1.5</code>	8192	1024	1.7 GB	English	Long records and wide vectors. Also needs <code>BETTERSEARCH_LOCAL_TRUST_REMOTE_CODE=1</code> .
<code>BAAI/bge-m3</code>	8192	1024	2.3 GB	Multilingual — its card claims 100+ working languages	A catalogue that is not in English. The largest download here.

The two that need `BETTERSEARCH_LOCAL_TRUST_REMOTE_CODE=1` ship their own modelling code, which `transformers` will only execute if you opt in. It is off by default because it runs code from the model repository on your machine.

Swapping one in. Set `BETTERSEARCH_LOCAL_MODEL`, then rebuild the index — the swap is not finished until you do. Every vector is stored under a `model_id` of `name@dimensions`, and both writing and querying check it, so a mismatch raises `ModelMismatchError` rather than returning confident nonsense from two incompatible number spaces. `--force` does not get around this; it only skips the unchanged-content check. Delete the two index files, or point `BETTERSEARCH_INDEX_PATH` somewhere new:

```
export BETTERSEARCH_LOCAL_MODEL=BAAI/bge-small-en-v1.5
rm .bettersearch/index.npz .bettersearch/index.json # or use a fresh path
bettersearch ingest --corpus data/catalogue_real.json
bettersearch evaluate --queries data/eval_real.json
```

Whether a swap was worth it is `bettersearch evaluate`'s question, not the model card's — it scores every mode on the labelled queries, so you can compare before and after on your own corpus rather than on someone else's benchmark. Two cautions when you read the result. Watch `truncated_chunks` from the ingest as well as the scores, since a longer window only pays where your text is actually long — on both corpora here, nothing reaches even half the cap. And a model that was not among the arms used to build the judged pool is

systematically under-credited: that is not a hypothetical, it is [what nearly sent this repository to the wrong conclusion](#).

What actually breaks at scale

The embedding model is not the bottleneck people expect. Four other things are. Three have a specific countermeasure in the code; the fourth was found by measuring a real 4,000-record catalogue and is not fixed yet.

1. Changing embedding model invalidates the whole index

Vectors from two models occupy different spaces. Comparing them returns confident nonsense — the worst kind of bug, because nothing errors.

Every vector is stored with its `model_id` and `dims`, and both index backends raise `ModelMismatchError` rather than answer a query embedded by a different model. That column is also what makes a *rolling* re-embed possible: without it, changing model means taking search offline.

2. Re-ingesting must not re-embed everything

Every chunk carries a `content_hash` over its embed text. Ingestion skips hashes already indexed, so a content update costs only the changed chunks. Re-run `bettersearch ingest` twice — the second run reports zero embedded.

3. Vector storage is the real wall

At 1M chunks × 1536 dims × 4 bytes you are holding **5.72 GB** of raw vectors before index overhead. Three multiplicative fixes, all implemented:

LEVER	EFFECT
Matryoshka truncation (1536 → 512)	3× smaller
<code>halfvec</code> fp16 storage	2× smaller
HNSW rather than flat scan	sub-linear query time

Together that is 6x: **5.72 GB** → **0.95 GB**, the difference between needing a large managed Postgres instance and fitting comfortably on a small one.

Self-hosting changes the arithmetic again, in both directions. The default encoder here is `bge-base-en-v1.5` at 768 dimensions: 1.43 GB for 1M chunks at fp16, comfortably under the managed-service figure and with no per-token cost. Dropping to `all-MiniLM-L6-v2` at 384 dims halves that again to 0.72 GB — at a measured cost of 0.079 nDCG, which is the trade to make deliberately rather than by default. `BETTERSEARCH_LOCAL_MODEL` is the whole switch.

4. An absolute relevance threshold does not survive a real corpus

`api/catalogue.py` cut meaning-based results at a fixed `RELEVANCE_FLOOR = 0.15`, tuned by eye on 74 records. Measured against 4,000 real ones with the same 41 queries, it stopped being a threshold at all:

RESULTS RETURNED PER QUERY	MIN	MEDIAN	P90	MAX
absolute floor, 0.15	116	763	3,262	3,665
relative: $\geq 0.90 \times \text{best}$	1	4	14	20
relative: $\geq 0.85 \times \text{best}$	2	9	27	41
largest-gap cut	1	2	6	24

books like Agatha Christie returns **3,665 of 4,000 records** above the floor — 92% of the catalogue, presented to a reader as "3,665 results". The count is worse than useless: it is confidently wrong.

The reason is that a fixed cosine threshold admits a roughly constant *fraction* of any corpus, not a constant number — 31% of the 74-record catalogue, 19% of the 4,000-record one — so the absolute count grows with the collection while the threshold looks unchanged. It scales exactly the wrong way, and nothing errors. This is the same brittleness that showed up when renaming one author moved a headline query from 8 results to 6: a single global constant cannot carry this.

The obvious fix is a relative cut, and $\geq 0.85 \times \text{best}$ is the plausible candidate on these numbers. It was not applied, because which cut is *right* is a question about relevance, and that needs the judged eval set rather than a count that looks sensible. Sane-looking output is not evidence — and when the judgements arrived they picked something else entirely. See *Settling the relevance floor* below.

5. Scaling while staying self-hosted

Self-hosted embeddings are the default path here: no per-token cost, no vendor dependency, and no content leaving your infrastructure — which matters more than the money if the corpus is ever commercially or personally sensitive.

Measure before you optimise. On this content the much-discussed 256-token cap of the older MiniLM default cost exactly nothing — the current default reads 512, so the headroom is larger still:

```
$ # chunk length as all-MiniLM-L6-v2 itself tokenises it
  seed corpus (112 chunks)      min 75  median 87  p90 98  max 115
  catalogue   (74 chunks)      min 17  median 29  p90 49  max 57
  chunks exceeding the 256 cap: 0 (0%)
```

Measured with the model's own tokeniser, not the chunker's word-based estimate — the cap is the model's, so the model's count is the one that decides. Both corpora are short enough that no chunk reaches even half the cap, let alone the 450-token chunk target, so MiniLM truncates nothing. `bettersearch ingest` reports `truncated_chunks` precisely so this is a measurement rather than an assumption — check it against *your* content before changing anything.

The cap is architectural, not a compute limit. MiniLM stops at 256 tokens because its positional embeddings end there. More CPU or a bigger GPU makes it faster, never able to read more. If your chunks *do* exceed it, there are two fixes and only one of them costs anything:

1. **Shrink the chunks** — `BETTERSEARCH_CHUNK_TOKENS=200`. Free, no model change. Costs you more chunks (bigger index) and more fragmented context.
2. **Swap the model** — one env var, no code change. Costs compute and RAM.

```
export BETTERSEARCH_LOCAL_MODEL="BAAI/bge-base-en-v1.5"
bettersearch ingest --corpus data/corpus_seed.json # re-embeds under the new model
```

Dimensions and the input limit are read off the loaded model, so the index, the truncation warning and the `model_id` guard all follow automatically.

MODEL	INPUT	DIMS	SIZE	NOTES
all-MiniLM-L6-v2	256	384	80MB	current default
BAAI/bge-small-en-v1.5	512	384	130MB	2× window, same index size
BAAI/bge-base-en-v1.5	512	768	440MB	2× index size
intfloat/e5-large-v2	512	1024	1.3GB	
nomic-ai/nomic-embed-text-v1.5	8192	768	550MB	needs <code>BETTERSEARCH_LOCAL_TRUST_REMOTE_CODE=1</code>
Alibaba-NLP/gte-large-en-v1.5	8192	1024	1.7GB	needs <code>trust-remote-code</code>
BAAI/bge-m3	8192	1024	2.3GB	multilingual

Bigger is not automatically better. Measured on this corpus, the 512-token model scored *worse* than MiniLM (nDCG 0.851 vs 0.890) — because nothing was being truncated, so the longer window bought nothing while the model itself happened to suit short passages less well. Re-run `bettersearch evaluate` after any model change; that is what the eval harness is for.

Measured CPU throughput on an ordinary container (no GPU), for a one-off backfill:

MODEL	CHUNKS/SEC (CPU)	1M CHUNKS
all-MiniLM-L6-v2	~119	~2.3 hours
bge-small-en-v1.5	~74	~3.8 hours

A single modern GPU is roughly 20–50× that, turning a backfill into minutes. Note this is a **batch** cost paid once per re-embed; query time is one embedding per search and is negligible either way.

The real steady-state cost of self-hosting is **RAM per web worker** (~600MB resident for MiniLM, more for larger models), not throughput. On a small Render instance that is the constraint that bites first — the usual fix is a separate embedding worker rather than loading the model into every web process.

If you ever do want a hosted provider

`openai` and `voyage` backends exist behind the same interface and are selected with `BETTERSEARCH_EMBEDDING_PROVIDER`. **Neither has been exercised against a live API** — they are verified only as far as constructing, reporting correct dimensions, refusing to run without a key, and handling empty batches. Treat them as a starting point, not a supported path.

PROVIDER	1M CHUNKS (~400M TOKENS)	INPUT LIMIT
openai text-embedding-3-small	~\$8 (~\$4 batched)	8,191 tokens
voyage voyage-3.5-lite	~\$8	32,000 tokens

Voyage additionally supports asymmetric encoding (`input_type` of `document` vs `query`), which is a real recall gain and is wired in.

Measured on 4,000 real catalogue records

Everything above this point was measured on records I wrote. This section is measured on **4,000 real bibliographic records** from Open Library, against 41 queries I drafted and Shen reviewed, with relevance judged by Claude Haiku on a pooled candidate set. Build it with `scripts/fetch_openlibrary.py` and `scripts/build_eval_set.py`.

The pool is the top 20 from each of six lanes — BM25, MiniLM over thin records, MiniLM over enriched records, bge-base over thin records, and the depth-200 bge-reranker-base lane in both its plain and exact-match-promoted forms — unioned, shuffled and judged blind: **2,378 judgements over 41 queries**. Six rather than four because an arm that is measured but not pooled is penalised for everything it finds that the pooled lanes missed; see *Comparing embedding models* below, where that mistake reversed a conclusion, and *Going deeper*, where it did so again.

The pool only ever grows. Judgements are carried forward on (query text, doc_id), so adding a lane can widen relevant_doc_ids but never shrink it — otherwise every previously published figure would silently improve against a smaller denominator.

The three modes

MODE	RECALL@5	MRR@10	NDCG@10
keyword	0.191	0.633	0.443
semantic	0.255	0.725	0.579
hybrid	0.220	0.710	0.549

Hybrid still loses, and that kills a hypothesis. The earlier result — hybrid below pure semantic — was explained away as an artefact of an eval set weighted towards keyword-hostile phrasing, with the prediction that real catalogue traffic, full of exact-name lookups, would be where fusion finally earned its place. It did not. On real records with five deliberate exact-title and exact-author controls, hybrid still sits below semantic. The earlier explanation was wrong, or at least incomplete.

The eval is not saturated. MRR@10 is 0.725, not the 1.000 that made the previous query set useless. There is room to improve and room to regress, which is the only condition under which a number means anything.

Enrichment, tested fairly for the first time

Enrichment on real records, Haiku 4.5, all 4,000:

ARM	RECALL@5	MRR@10	NDCG@10
thin records	0.264	0.725	0.579
enriched	0.307	0.793	0.671

+0.092 nDCG, a 16% relative gain, improving 26 of 41 queries. Far more modest than the synthetic corpus suggested, and far more believable. Biggest gains are where a title says nothing about content:

QUERY	THIN	ENRICHED
cycling long distances	0.366	0.747
Joy of Cooking	0.631	1.000
a novel where the beekeeper is the detective	0.307	0.613
an uplifting story after a hard year	0.073	0.344
second world war in the pacific	0.073	0.290

Enrichment is not free, and the failures are systematic. Five queries got worse, and the worst is instructive:

QUERY	THIN	ENRICHED
Sue Monk Kidd	1.000	0.387
what to do about money worries	1.000	0.695

Sue Monk Kidd is an exact-author lookup with one matching record. Enrichment surrounds that record with topical prose about bees, race and the American South, and the author's name stops dominating its vector. **Enrichment helps queries about aboutness and hurts queries about identity** — which is an argument for routing by query type, not for enriching everything and hoping.

It also reintroduces a problem the project had written off. Enrichment takes the median record from 21 words to 117, and **79 chunks now exceed MiniLM's 256-token cap** where thin records exceeded it zero times. The earlier finding that the cap "costs exactly nothing" was true only of thin records. Two decisions each measured in isolation interact, and Opus enrichment — 74-word synopses against Haiku's 44 — would truncate more.

Settling the relevance floor

RELEVANCE_FLOOR = 0.15 was tuned by eye on 74 records. Against real judgements, on the thin 4,000-record arm:

STRATEGY	PRECISION	RECALL	F1	MEDIAN RESULTS
absolute 0.15 (old)	0.042	0.977	0.080	763
relative $\geq 0.90 \times$ best	0.585	0.273	0.372	4
relative $\geq 0.85 \times$ best	0.515	0.356	0.421	9
largest-gap cut	0.653	0.223	0.333	2
fixed top-5	0.527	0.255	0.344	5
fixed top-10	0.459	0.364	0.406	10
fixed top-15	0.416	0.483	0.447	15
fixed top-20	0.396	0.573	0.469	20

Precision **0.042**: the floor admits almost everything and calls it a result set. But the fix I proposed from result counts alone — a relative cut — is **not** the winner. Plain top-k beats every threshold on F1, because thresholds buy precision by throwing away recall the pagination would have handled anyway. Judging by counts that "look sensible" would have picked the wrong answer; this is why the fix waited for the judgements.

Where the evidence stops. The eval pooled each lane to depth 20, so no record below rank 20 has a judgement and every one of them scores as irrelevant. F1 is still *rising* at k=20 — 0.344, 0.406, 0.447, 0.469 across the sweep — and the fall after it (0.464 at 25, 0.369 at 50) is an artefact of running past the pool, not a peak. So the honest claim is narrow: **20 is the deepest cut this eval can vouch for, and it beats everything shallower.** Whether 30 would be better is unmeasured, and would need a deeper pool to answer.

Re-derived after the pool grew to six lanes and 2,378 judgements: every F1 fell by about 0.01, because recall is measured against a larger set of relevant records, and **the ranking of the strategies did not change at all.** fixed top-20 still wins on both arms (0.469 thin, 0.520 enriched), so SEMANTIC_TOP_K = 20 stands unaltered. A re-pool that moves every number and no decision is the good case.

api/catalogue.py now retrieves SEMANTIC_TOP_K = 20 directly instead of retrieving everything and filtering. books like Agatha Christie returns 20 results rather than 3,665. Reproduce the whole table, both arms and the sweep:

```
python scripts/tune_cutoff.py thin=.bettersearch/real \
    enriched=.bettersearch/real-enriched
```

And 20 is not a universal constant. It is measured for a 4,000-record catalogue. On the 74-record demonstration catalogue at the top of this file it is more than a quarter of the shelf, and results 11 to 20 for the headline query are a Punjabi book with no title and a DVD about a lighthouse — the same error as the floor it replaced, a number tuned on one corpus applied to another. The difference is that this one is labelled and settable (`BETTERSEARCH_TOP_K`) rather than buried, and no second number has been invented for the small catalogue: there are no judgements for it, and guessing one is how the first version of this went wrong.

One visible consequence: facet counts in the sidebar now describe the 20 results returned, where before they described a 763-record set the reader could never page to. The keyword lane is untouched and still returns full depth — BM25 stops on its own, since a record sharing no term with the query does not match at all, while cosine similarity scores every record against every query and needs the list ended for it.

Comparing embedding models

`all-MiniLM-L6-v2` is 384 dimensions and reads 256 tokens. Swapping it is a config change — `BETTERSEARCH_LOCAL_MODEL` — so the obvious question is whether the retrieval ceiling is the encoder or the records. Against `bge-base-en-v1.5` (768 dimensions, 512 tokens, still self-hosted, still no API key):

	THIN RECORDS	ENRICHED RECORDS
MiniLM (384d)	0.579	0.671
bge-small (384d) — <i>the browser page until Sept 2026</i>	0.610	not built
bge-base (768d) — <i>the server and the browser page</i>	0.658	0.743

nDCG@10 on the four-lane pool. `bge-small` was measured because it was what `docs/catalogue-standalone.html` ran, and a page should not quote figures from a model it is not using. The page now runs `bge-base`, so the 0.658 row describes both — but the small row stays, because it is the measurement that made the 0.048 gap concrete enough to act on. It sits where its size suggests: better than MiniLM, short of bge-base. Note it was **not** in the judging pool, so by the argument below it is if anything under-credited.

They stack, and almost additively. Enrichment is worth +0.092 over the baseline, the encoder +0.079, and both together +0.164 against the 0.171 you would get if they never overlapped. Best arm improves 29 of 41 queries and regresses 3.

The practical reading: **a bigger encoder buys most of what LLM enrichment buys, for free.** bge on thin records (0.658) nearly matches MiniLM on enriched records (0.671) — no API calls, no per-record cost, no vendor. Enrichment is still worth its £7 on top, but it is the second thing to try, not the first. The 512-token window also removes the truncation the 256-token cap was causing: 0 truncated chunks against 79.

What it costs: vectors are twice the size, so the storage arithmetic above doubles, and embedding is **6× slower on CPU** — 400 records in 10.0s against MiniLM's 1.6s. That is an ingest cost, paid once per record, not a query cost.

I nearly got this exactly backwards. The first comparison said bge was *worse* — 0.578 against 0.588 — and I was ready to report that the records were the ceiling. Two things were wrong with it.

The first was a red herring: bge wants a query instruction prefix for retrieval and our provider adds none. Adding it changed nothing (0.572).

The second was the real fault. `data/eval_real.json` was pooled from three lanes, all MiniLM or BM25. **Judging only what the pooled lanes retrieve means any record a new system finds that they missed is unjudged, and unjudged counts as irrelevant.** Measured: 100% of MiniLM's top 10 had been seen by the judge against

71% of bge's. Re-pooling with bge as a fourth lane added 312 records no other lane had ever surfaced, and moved bge by **+0.080** — the entire difference between "slightly worse" and "close to enrichment".

This is the same failure the enrichment arm was protected from by pooling all three lanes, arriving in a new costume. The fix is in the tooling: a lane and an arm can each name their own encoder, and `scripts/compare_arms.py` says in its docstring that an unpooled arm cannot be compared.

The combined arm is itself only 90% judged, so 0.743 is if anything an undercount — the bias runs against the conclusion, which is why it stands without a fifth re-pool.

One honest wrinkle: on `learning to be present`, the query that prompted all this, plain bge is the best arm. It returns *Be Here Now* at rank 1 where MiniLM returned *Introduction to Algorithms*; adding enrichment pushes it to rank 4. Enrichment wins on average and dilutes some queries, exactly as `Sue Monk Kidd` showed earlier.

Open: does the top model tier buy better retrieval?

Enrichment above is Haiku 4.5 on all 4,000 records. Opus 5 was run on a 289-record sample before the comparison was abandoned, and on those same records it is plainly the richer writer:

ON THE SAME 289 RECORDS	HAIKU 4.5	OPUS 5
<code>recognised: true</code>	52%	76%
synopsis words	39	65
topics	7.6	13.5

An earlier version of this table read 74% / 93% off the first 120 records. Those 120 are all detective and science fiction — the corpus is stored in the order it was fetched, subject by subject — and famous genre novels are the easiest thing in a catalogue for a model to recognise. Widening the sample to 289 drops both models by more than twenty points. The ordering between them survives; the absolute rates did not, and a sample drawn from the top of a subject-ordered file is not a sample.

This does not answer whether Opus retrieves better, and it was never going to. A valid Opus arm needs all 4,000 records enriched by Opus: with 289 enriched among 4,000 thin ones, the enriched records gain an advantage purely for being enriched. The sample supports a quality comparison, not a retrieval one. Buying the real answer costs about £37 of Opus enrichment against £7 for Haiku.

The 4k result argues against assuming richer is better. Enrichment already regressed five queries by diluting records — `Sue Monk Kidd` fell 1.000 → 0.387 because topical prose crowded out an exact-author match. Opus writes 68% more synopsis and 78% more topics, so it would dilute harder. Richer could plausibly retrieve *worse*, and the 79 chunks already pushed past MiniLM's 256-token cap would become more — though on bge's 512-token window that particular objection disappears, since the enriched records truncate to 0 chunks there.

The 289 Opus enrichments are committed at `data/enrichment_real_opus_sample.jsonl` so the quality figures above can be checked.

How much to trust this

- **Judge self-consistency 94%** (94 of 100 re-judged pairs identical). A 50-pair sample is in `data/eval_real_spotcheck.md` for a human to check.
- **Pooling bias.** Records no lane retrieved were never judged, so recall is relative to the pool, not absolute. All three measured lanes contributed — 309 records came only from the enriched lane, and judging before enriching would have scored every one of them as irrelevant.
- **Grades are an LLM's**, validated by sampling, not human ground truth.
- **Queries are author-drafted, reviewer-edited.** Not independently written.

- **Holdings are invented.** Bibliographic data is real; availability, branch and format are not.
- **Two queries score 0.000 on both arms** — `learning to grow vegetables in pots` and the one about a boy wizard at school. Reported rather than quietly dropped.

Narrowing: the filter has to come before the cut

Found while asking what else Vespa could offer. It is not a missing feature — it was a defect here.

`run_search` retrieved the semantic top 20 and *then* applied facet filters, so ticking "Large print" did not search the large-print records. It kept whichever of twenty already-chosen records happened to be large print. Measured over 41 queries × 10 facet values on the 4,000-record corpus, through the real pipeline:

	FILTER AFTER THE CUT	FILTER BEFORE IT
results returned	4.00	19.96
returned fewer than 3	31.2%	0.0%
returned zero	2.9%	0.0%

Those facets each cover 14–18% of the corpus. At a real library "available now at my branch" is a far smaller slice, and the facet panel would have returned almost nothing almost always.

The fix is a **candidate pool**: retrieval fetches `SEMANTIC_POOL` (200), the filter selects within it, and the cut to `SEMANTIC_TOP_K` comes afterwards. The measured decision does not move — `scripts/tune_cutoff.py` chose 20 as *what a reader sees*, and a reader still sees 20. The pool is never displayed in full, so this is not the absolute floor that admitted 3,665 of 4,000 records.

The regression gate, checked before anything else. An unfiltered search must be untouched: `nDCG@10` stayed at exactly **0.6582**, with **41/41 identical result lists**. On the books-and-events corpus it is 39/41 — the two that differ had been returning 18 results because a weekly event series collapsed inside the old top 20, and now return a full 20. The additions land at ranks 19–20, so `nDCG@10` is unchanged at 0.6511 there too. A strict improvement, observed rather than discovered later.

What moved, and is a real cost. Facet counts are now computed over the pool rather than the page, which is forced: counting over a filtered retrieval would collapse the sidebar to the value already selected and make cross-narrowing impossible. So a facet can read a larger number than the list it opens — "Large print 28" above a list of 20. The count is still honest, because every record it counts is reachable by clicking it. That was *not* true of the 763-record shadow set the old floor produced, which is the thing this must not drift back into.

Not claimed: `nDCG` over the filtered searches rises 0.2759 → 0.3186, and that number should not be quoted. Filter-aware retrieval surfaces records the four pooled lanes never saw, so judged coverage falls and the comparison is subject to exactly the bias described above. The collapse in result *count* is arithmetic and needs no judgements at all.

`200` is judgement, not measurement, and the constant says so. It is set so a facet covering a seventh of the corpus still offers comfortably more candidates than the twenty displayed, and `BETTERSEARCH_POOL` overrides it. Raising it cannot change an unfiltered search.

Reranking: the ordering is the problem, not the retrieval

Asked because the engineering team is moving to Vespa, whose contribution to *quality* — as opposed to serving — is its reranking phase. The ceiling was measured before anything was built. Reordering only the candidates

already retrieved, with every relevant one moved to the top:

DEPTH	OURS	ORACLE	HEADROOM	JUDGED COVERAGE
20	0.6582	0.9124	+0.254	100%
50	0.6582	0.9703	+0.312	92.7%
100	0.6582	0.9850	+0.327	98.8%
200	0.6582	1.0000	+0.342	100%

The retrieval is fine; the ordering is the problem. 161 records across the 41 queries are judged relevant, retrieved, and sitting at ranks 11–20 — on page two. Per-query headroom has a median of 0.209, so it is broad rather than a few outliers; the seven queries with nothing to gain are exact-name controls, already at 1.000.

Coverage is high at every depth here **because the pool was rebuilt to make it so**. It was not always: the reranked lane at depth 200 was 59.5% judged, and the figures that produced are in [Going deeper](#) below as a worked example of how badly that misleads. Reranking does not change the candidate set, so coverage at the *display* depth of 20 was always 100% by construction — `scripts/tune_reranking.py` asserts that rather than trusting it. Deeper, it had to be bought.

Rerank depth 20 — the twenty candidates the page displays. Judged coverage is 100% on every arm, so nDCG@10 is comparable down the column.

ARM	NDCG@10	VS BASE	RECALL@10	CONTROLS	MS/QUERY
baseline (bge-base, as shipped)	0.6582	—	0.4196	1.000	0
+ BM25 as a weighted second phase	0.6782	+0.020	0.4258	1.000	0
+ <code>ms-marco-MiniLM-L-6-v2</code> over thin records	0.6632	+0.005	0.4263	1.000	186
+ <code>ms-marco-MiniLM-L-6-v2</code> over enriched records	0.7149	+0.057	0.4780	0.877	356
... + exact-match promotion	0.7280	+0.070	0.4780	1.000	356
+ <code>bge-reranker-base</code> over thin records	0.6573	−0.001	0.4277	1.000	890
+ <code>bge-reranker-base</code> over enriched records	0.7257	+0.068	0.4871	0.926	2,809
... + exact-match promotion	0.7308	+0.073	0.4852	1.000	2,809
oracle (the ceiling)	0.9124	+0.254	0.5560	1.000	—

A cross-encoder is worth nothing on a catalogue record. Both rerankers are flat on the records as they ship — +0.005 and −0.001. That is not a weak model: on a written sentence `bge-reranker-base` separates cleanly (0.643 for a beekeeping passage against 0.000 for a calculus textbook). It is that the record is a 24-word metadata stub — a title, an author, a publisher, a semicolon list of subject headings — and a model trained to read passages has nothing to read. Give it the enriched text instead, at 95 words median, and the same model is worth **+0.073**.

So reranking does not replace enrichment; it is **unlocked** by it. The two are the same bet — that these records are too thin — arriving from different ends.

And it broke the thing libraries care most about. Both enriched arms regressed the known-item controls, `Sue Monk Kidd` worst at −0.369. That is the *same* failure enrichment itself has, recorded above: it helps queries about aboutness and hurts queries about identity. The reranker inherits it, because it is reasoning over the same prose.

The fix was already in the repository. `src/bettersearch/exact.py` lifts records the reader arguably *named* by rule rather than by score, and its docstring calls it "a guarantee rather than an improvement", worth 0.658 → 0.658.

Against a reranker it starts earning its keep: controls back to **1.000**, and the mean goes *up* rather than down, because the queries it rescues were the ones being broken. A component measured as worthless becomes valuable the moment something upstream is strong enough to be unpredictable.

Going deeper: reranking pays at 20 and decays with depth

"Retrieve wide, rank deep" is what Vespa's phased ranking exists for, and it looked like the natural next move, because the pool we already fetch holds far more of the good records than the page we show:

RETRIEVAL DEPTH	RECALL OF JUDGED-RELEVANT	MISSED
10	0.420	501
20 — what we display	0.628	340
50	0.809	185
200 — the pool	0.961	48

340 judged-relevant records are retrieved and thrown away on every run. So the reranker was given the full pool instead of the top 20. **It does not help. It decays, monotonically, all the way down:**

RERANK DEPTH	20	50	100	200
recall@10 vs baseline	+0.066	+0.047	+0.035	+0.010
judged coverage	100%	92.7%	98.8%	100%
ms/query (CPU)	2,809	5,548	10,093	19,937

Seven times the latency for a seventh of the gain. **Rerank the twenty you display, and stop there.** For a Vespa `rerank-count`, that is the number.

Depth also exposes something depth 20 hides. On records **as they ship**, a cross-encoder stops being merely useless and turns harmful: **-0.017** for MiniLM and **-0.040** for bge at depth 200, against +0.005 and -0.001 at depth 20. Given a hundred 24-word stubs instead of twenty, a passage model reorders them actively worse than retrieval did. The need for prose is not a nice-to-have that buys a bit more — without it, going deeper costs you.

This took two wrong answers to reach, and both are instructive

The first sweep, before the pool was rebuilt, read `+0.079 / +0.029 / -0.007 / -0.062` with coverage falling to 59.5%. That looked like the pooling penalty and it was: reranking 200 candidates promotes records no judge ever saw, and every one scores zero however good it is.

So it was re-run with `--judged-only`, which discards unjudged candidates and pins coverage at 100%. That read `+0.079 / +0.072 / +0.091 / +0.087` — apparently no decay at all, strongest at 100 — and it was recorded here as the true shape the eval could not see.

It was the larger error of the two. Discarding unjudged candidates does not neutralise the bias, it reverses it: what gets discarded is precisely the junk the reranker promoted and nobody graded, so the arm is scored only on the candidates where it was already known to be doing well. It flattered the reranker roughly ninefold at depth 200 — `+0.087` against a judged truth of `+0.010`.

The fix was to pay for the judgements. `data/rankings/` holds the depth-200 reranked lane, both plain and exact-match-promoted; both were pooled and the 399 records they surfaced that no judge had seen were graded, at a cost of \$0.06. 81 came back relevant. Coverage at depth 200 went 59.5% → 100%, and the number stopped moving.

That account makes a prediction, so it was tested rather than asserted. If `--judged-only` is biased by exactly the coverage it discards, it must agree with the judged truth wherever coverage is already 100%, and over-

read everywhere else. Both sweeps, all four depths, against the re-pooled eval:

RERANK DEPTH	BGE + PROMOTION — THE LANE WE POOLED			MINILM + PROMOTION — NOT POOLED		
	judged truth	--judged- only	gap	judged truth	--judged- only	gap
20	+0.0657	+0.0657	0.0000	+0.0584	+0.0584	0.0000
50	+0.0465	+0.0596	+0.0131	+0.0308	+0.0680	+0.0372
100	+0.0349	+0.0384	+0.0035	+0.0292	+0.0458	+0.0166
200	+0.0097	+0.0097	0.0000	+0.0249	+0.0419	+0.0170

It agrees **to four decimals at exactly the two places it should**, and nowhere else. Depth 20, where every candidate was judged anyway. And depth 200 for the bge arm — *because that is the lane whose candidates were pooled and judged*. MiniLM, same corpus and same depth but never pooled, still over-reads there by +0.017.

The arm we paid for converges; the arm we did not, does not. That is about as close to a controlled experiment as this eval allows.

Two general lessons, both cheaper to learn here than in production:

- **A coverage-correction that changes the answer is not a correction.** Both `--judged-only` and raw pooling are biased estimators; agreeing with neither is what the judgements are for. `scripts/bound_repool_effect.py` now exists to bound the gap before paying, and its own prediction was 3× out on magnitude — it assigns relevance independently of rank, and relevance correlates with rank, so it over-separates arms. Its docstring says so with the numbers.
- **recall@10 is not pooling-safe.** I proposed the original sweep claiming it was, because the denominator is fixed. The *numerator* is not: an unjudged record entering the top 10 displaces a judged-relevant one. The script says so where the claim was made.

This one ships

`ms-marco-MiniLM-L-6-v2` over enriched records, depth 20, with exact-match promotion after it, is wired into `api/catalogue.py` and on by default. `src/bettersearch/rerank.py` holds it; `BETTERSEARCH_RERANK=` turns it off.

Measured through the serving path, not the script — `run_search` scored against the same 41 judged queries:

PATH	NDCG@10	RECALL@10	CONTROLS
retrieval only	0.6582	0.4196	1.000
+ rerank, no promotion	0.7149	0.4780	0.877
+ rerank + promotion	0.7280	0.4780	1.000

Identical to the script's figures to four decimals, which is the point of running it this way: the number in this README now describes the thing a reader actually gets.

Three things the measurement did not cover, which shipping had to.

Order matters. Reranking runs **before** promotion. Reversed, the reranker pushes the named records promotion just lifted straight back down, and the controls sit at 0.877 — the middle row above is what that looks like.

A bare record is not worth reranking. Records with no enriched prose keep their retrieval position rather than being scored on a 24-word stub, so a catalogue of books beside un-enriched events reranks the half that benefits. Point the service at a catalogue with no enrichment at all and it says so on startup rather than no-opping silently.

The score stays a cosine. A cross-encoder emits an uncalibrated logit; letting it reach the response would redefine a published field. So `score` is still the retrieval score and no longer descends with rank — the card carries `"reranked": true` to say which ordering it is looking at.

The cost, measured through a running server: +371 ms median per search — 184 ms without reranking against 555 ms with, over the same twelve queries on the same box, warm. An earlier in-process measurement said +104 ms; it was measured against a different baseline and was wrong. Render's hardware is different again, so treat 371 ms as this box's number rather than the service's.

And the model must be warmed at startup, which is not a detail. Left to load on the first search — the obvious lazy design, and what `api/catalogue.py` still does so the test suite needs no model — the first search after boot took **85 seconds** on a container with the weights already cached. That is a Hub round-trip plus construction, not a download, so fetching the model during the build does not avoid it. `api/main.py` loads it, runs one throwaway inference to pay torch's warmup, and builds the enrichment store before the health check passes.

One thing reranking is *not* responsible for: the first semantic search is still several seconds, because `_closest_headings` embeds the corpus's subject headings on first use. Measured with reranking turned off it is **8.1 s**, which is slower than with it on. That cost predates this change and is a separate thing to fix.

And the demo page cannot do this. A second model plus one inference per candidate does not fit in a file meant to open from an email, already ~7 MB against an 8 MB ceiling. So the standalone page shows *retrieval* and the service adds a ranking phase on top; `scripts/check_browser_parity.py` compares the two retrieval layers and says so in its docstring. The divergence is stated rather than discovered, which is the lesson from the page once running `bge-small` against a `bge-base` server.

What is not claimed. The best arm captures 29% of the headroom; 0.18 of nDCG is still on the table and unexplained. `bge-reranker-base` costs 2.8 seconds per query on CPU for 20 candidates — unusable without ONNX or a GPU, and MiniLM gets most of the gain for **13% of the cost**: 96% of it on nDCG@10 (+0.070 against +0.073) and 89% on recall@10 (+0.058 against +0.066), which is why it is the one wired in. The BM25 second phase at +0.020 is reported, not shipped: `w` was swept on the same 41 queries it is scored on, so that number is an upper bound on the idea rather than an estimate of it. Forty-one queries remains few, and the ceiling is still a set of LLM judgements — a reranker approaching the oracle is increasingly agreeing with Claude Haiku, not demonstrably serving readers better.

Enrichment for thin catalogue records

The production target is a library catalogue of 100k–1M items holding **thin records only** — title, author, subject headings. Around 15 words per item. That is not enough surface to embed: a catalogue record names a shelf position, while semantic search needs text that names *concepts*.

The enrichment pipeline generates that text with an LLM, and can state concepts the record never contains — turning `Subjects: Great Britain -- History` into prose mentioning the Norman Conquest, William the Conqueror and the Domesday survey. It builds the vocabulary bridge deliberately instead of hoping for it.

How much does thinness actually cost? (measured)

`scripts/synthesise_catalogue.py` strips the seed corpus down to thin records — title, synthetic author, LCSH-style subject headings, **no body text at all** — so the same 32 labelled queries can be re-run against a realistically impoverished corpus, then enriched and re-run a third time.

ARM	CORPUS	RECALL@5	MRR@10	NDCG@10
Full text	full article text	0.841	0.984	0.890
Baseline	thin records only	0.718	0.917	0.789
Treatment	enriched records	0.836	1.000	0.894

Enrichment recovers **104% of the gap** between thin records and full text, and improves 24 of the 32 queries. The per-query view is where the case is actually made — run `python scripts/compare_arms.py ceiling=... thin=... enriched=...`:

QUERY	FULL TEXT	THIN	ENRICHED
planning a landing on a defended coast	0.871	0.296	0.626
1066	0.876	0.369	0.706
how electricity gets from a power station...	1.000	0.581	0.884
how plants make food from sunlight	0.761	0.503	0.956
drugs that stop working	0.920	0.693	1.000

Queries collapse when the relevant documents' titles do not name the concept, and that is exactly what enrichment repairs. 1066 falls by more than half on thin records because *The Domesday Book*, *The Bayeux Tapestry*, *Stamford Bridge* and *Harold Godwinson* never contain the string — enrichment puts it there, and the query recovers to 0.706.

Four things that keep this honest

"Full text" is not a ceiling, and enrichment passes it. 0.894 against 0.890, beating it on 17 of 32 queries. That is not a paradox: an article was written to be read, while enrichment is generated to be *retrieved* — the `questions` field puts "What happened at the Battle of Hastings?" into the embedded text, and query-to-question is a closer shape match than query-to-document. The row is relabelled from "Ceiling" accordingly.

MRR@10 hit 1.000 — the eval set is saturated. A relevant record now ranks first for all 32 queries, so this query set can no longer distinguish further improvement. Any next experiment needs harder queries or a larger corpus; do not read 1.000 as headroom.

Enrichment made 5 of 32 queries worse, and it is not free. `machine that broke German codes in the war` fell 0.832 → 0.613: adding topical prose can dilute a record that already matched well. The aggregate hides this — check the per-query table before assuming enrichment is a pure win.

Every record came back recognised: false (0 of 112). The authors are invented, so the model correctly declined to claim knowledge of the specific works and used grounded expansion throughout — subject headings turned into prose, nothing fabricated. So these numbers measure the **conservative mode only**. The knowledge-synopsis path is still unmeasured, and on a real catalogue of real books it would carry a meaningful share of the corpus.

Design

Three decisions carry the risk:

The generated text is a retrieval bridge and is never displayed. Match against it, then show the real catalogue record. A library is an authority-bearing institution; this one rule demotes a hallucination from a false statement by the library to a mildly bad search result.

Two modes, chosen per item by the model. Thin records maximise hallucination exposure — given only a title and a subject heading, the model draws on training knowledge rather than the record. So it first reports whether it

actually knows the work:

RECOGNISED	MODE	MAY USE
true	Knowledge synopsis	Its own knowledge of the work. Rich.
false	Grounded expansion	Only the record. Expand subject headings; invent nothing.

Grounded expansion is the safety floor and is still worth the call — turning authoritative controlled vocabulary into searchable prose is the bridge we want, with no fabrication risk. `bettersearch enrich` reports the split.

Enrichment is added alongside the record, never instead of it. The raw record stays indexed so the keyword lane can still match exact author, title and subject-heading terms. This is where `hybrid` should finally beat pure `semantic`, unlike on the article corpus above — real catalogue traffic is full of exact-name lookups, which is what BM25 is for. Re-measure rather than carrying the earlier conclusion across.

Prompt version is tracked like model id. A prompt change invalidates every stored enrichment, exactly as an embedding-model change invalidates every vector — except regenerating costs money rather than CPU. Every record stores `prompt_version`, `enrichment_model` and a `source_hash` of the input, so re-runs are incremental and a prompt edit is a costed, staged migration.

Running it

```
export ANTHROPIC_API_KEY=...
python scripts/synthesise_catalogue.py          # or bring your own catalogue
bettersearch enrich --corpus data/catalogue_thin.json --index
```

Reproducing the table above needs no API key. The 112 enrichments from that run are committed at `data/enrichment_seed.jsonl` — copy them into the store and every number is re-derivable offline:

```
mkdir -p .bettersearch && cp data/enrichment_seed.jsonl .bettersearch/enrichment.jsonl
BETTERSEARCH_INDEX_PATH=.bettersearch/arm-ceiling bettersearch ingest --corpus data/corpus_seed.json
BETTERSEARCH_INDEX_PATH=.bettersearch/arm-thin bettersearch ingest --corpus data/catalogue_thin.json
BETTERSEARCH_INDEX_PATH=.bettersearch/arm-enriched bettersearch enrich \
  --corpus data/catalogue_thin.json --index      # store is warm: no API calls
python scripts/compare_arms.py ceiling=.bettersearch/arm-ceiling \
  thin=.bettersearch/arm-thin enriched=.bettersearch/arm-enriched
```

Uses the Batch API by default (half price, results keyed by `custom_id` since they arrive in arbitrary order).

Resumable: everything already stored for this prompt/model/source is skipped, so a crash costs only the in-flight batch. `--sync` switches to one request per item for small runs.

Cost

Thin records invert the usual shape — the record itself is ~26 tokens against a cached system prompt, and output is the bulk — so **synopsis length is the primary lever, not model tier**.

Measured on the 112-record run above (Opus 5, Batch API):

uncached input	6,353	cache writes	84,392	cache reads	38,360
output	29,353	mean	262 tokens/record		
total spend	\$0.66	cache hit rate	31%		

The 31% cache hit rate is the finding worth carrying forward. A batch submits every request at once, so they race on the system-prompt cache and most *write* it rather than read it. That pushed input to 44% of the bill — not the rounding error "output dominates" implies. At catalogue scale a sustained run keeps the prefix warm and the

ratio improves, but this is exactly the silent cost leak `client.py` warns about, so check `cache_read_input_tokens` on any real run before trusting a projection.

Projections below assume caching works properly; at the 31% hit rate measured here the Opus figure is nearer \$5,900 than \$4,000.

MODEL	1M ITEMS (BATCHED)	100K ITEMS
Haiku 4.5	~\$800	~\$80
Sonnet 5	~\$1,600	~\$160
Opus 5 (default)	~\$4,000	~\$400

At 100k the tier barely matters. At 1M it is a real decision — so measure it rather than guessing. Enrich a few hundred items at each tier into the same store (the model is part of the cache key, so tiers coexist) and compare with `scripts/compare_arms.py`. Embedding stays local and free either way.

Using the pgvector backend

```
docker compose up -d
export BETTERSEARCH_DATABASE_URL="postgresql://bettersearch:bettersearch@localhost:5433/bettersearch"
export BETTERSEARCH_INDEX=pgvector
bettersearch ingest --corpus data/corpus_seed.json
bettersearch compare "Norman conquest"
```

Results should be identical to the numpy backend. The schema is created on first write and includes a generated `tsvector` column with a GIN index — the query to switch to when the corpus outgrows loading every chunk into memory.

HTTP API

ENDPOINT	PURPOSE
GET <code>/health</code>	provider, backend, chunk count, <code>model_id</code>
POST <code>/search</code>	<code>{query, mode, top_k}</code> → ranked results
POST <code>/compare</code>	one query, all three modes

Responses follow `{"status": "success", "data": {...}}`.

Dropping this into a Django/DRF project

The core library has no framework dependency, so the whole integration is one view:

```
# search/views.py
from bettersearch import Searcher
from rest_framework.response import Response
from rest_framework.views import APIView

_searcher = Searcher()          # module-level: loads the model once

class SemanticSearchView(APIView):
    def post(self, request):
        query = request.data.get("query", "").strip()
        if not query:
            return Response({"status": "error", "message": "query is required"}, status=400)
        response = _searcher.search(
            query,
            mode=request.data.get("mode", "hybrid"),
            top_k=int(request.data.get("top_k", 10)),
        )
        return Response({"status": "success", "data": response.to_dict()})
```

Point `BETTERSEARCH_DATABASE_URL` at the existing database and the chunk table lives alongside the rest of the schema. Embedding work belongs in a Celery task, not the request cycle.

Two versions of the box

The page ships in two interactions, built from one source by `scripts/build_standalone.py --interaction toggle|blended`, so a librarian can be shown both without the comparison being confounded by a different corpus or a different model.

<code>docs/catalogue-standalone.html</code>	the reader picks catalogue search or search by meaning
<code>docs/catalogue-blended.html</code>	one box; meaning, with named records lifted and labelled

Why a blend rather than a router. Two simpler answers were measured first and both failed. Reciprocal rank fusion scored **0.741 against semantic's 0.890**, because blending two full rankings drags a good list down with a mostly-empty one. Routing — classify the query, send it down one lane — turned out to have nothing to fix: meaning-based search wins *both* kinds of query, **103/113 against keyword's 99/113** on name lookups and **0.658 against 0.443** on need-shaped ones. `scripts/check_query_routing.py` has both.

So the blend keeps the meaning ranking as the spine and promotes records the reader arguably **named**, using the one signal neither lane can see: word order. BM25 is a bag of words and an embedding blurs proper nouns, so a phrase in sequence is free information. Three predicates, in [src/bettersearch/exact.py](#), none needing a model: an exact title, a run of three-or-more content words inside a title, or an author name.

The thresholds are measured. Against the 36 need-shaped eval queries and the 5 known-item controls beside them:

SIGNAL	FIRES ON NEEDS	CATCHES CONTROLS
2-word phrase in a title	11/36 — unusable	3/5
3-word phrase in a title	1/36	1/5
query equals an author name	0/36	3/5

What it is worth, stated honestly: not a relevance improvement. `nDCG@10` over the judged queries is 0.658 with it and without it, identical at every phrase length and every cap; name lookups go from 103/113 to 104/113 at rank 1. One query in 113 is noise. It earns its place by **labelling** — a card reading "by this author" is legible where

a cosine score is not — and by making a **guarantee**: a record whose exact title someone typed is on page one because a rule puts it there, not because the encoder happened to agree.

Held to the Python by `scripts/check_browser_parity.py --blend`: where both implementations return the same record they give it the same reason, **49/49**.

What runs where

	SERVER (<code>UVICORN API.MAIN:APP</code>)	THE STANDALONE FILE
retrieval	Python, numpy index	JavaScript, in the page
keyword	BM25 in <code>src/bettersearch/keyword.py</code>	the same BM25, ported
query embedding	sentence-transformers, fp32	transformers.js, ONNX int8
model	<code>bge-base-en-v1.5</code> , 768d	the same , <code>bge-base-en-v1.5</code> , 768d
corpus vectors	float32 in the index	int8 + per-vector scale, in the file
first use	model resident, ~1.1 GB	~110 MB download, then cached
needs a network	no	only to fetch the model, once

They now run the same encoder, so a query that ranks one way on the server ranks that way in the file you send someone. The only difference left is quantisation — int8 corpus vectors and a quantised ONNX model against fp32 — which is measured by `scripts/check_browser_parity.py` rather than assumed.

The page used to run `bge-small` on the argument that a browser could not be asked to download `bge-base`. The quantised build is 110 MB against 34 MB, so the argument was wrong about its own premise, and it cost more than it saved: the demo behaved differently from the service, silently, and was the weaker of the two.

Third-party licences

The page loads two things it does not ship:

- [transformers.js](#) — Apache-2.0, from jsDelivr.
- [BAAI/bge-base-en-v1.5](#) — MIT, via the ONNX build at `Xenova/bge-base-en-v1.5`, from the Hugging Face hub.

Both permit redistribution, so the file can be hosted or emailed freely. The bibliographic records are Open Library, public domain; the holdings and the events are invented, as the page footer says.

Scope

Deliberately **not** included: LLM answer generation, query expansion, a reranker, authentication, and multi-tenant scoping. This PoC answers one question — does retrieval by meaning beat retrieval by keyword on this content — and each of those additions would obscure the answer.

Tests

```
pytest
```

Covers chunk boundaries and overlap, hash stability, the incremental-ingest skip, model-mismatch refusal, BM25 ranking, RRF ordering, numpy top-k against hand-computed values, and the full enrichment pipeline — resumability, prompt invalidation, per-model isolation, and out-of-order batch results. The tests use a deterministic fake embedding provider and a fake enrichment client, so they need no model, no network and no API key.

